

Programação de Sistemas

Revisão da Linguagem C Padrão - Módulo 6 -

Prof. Ronaldo Augusto de Lara Gonçalves

Agradecimentos ao Prof. Marcelo de G. Malheiros

- Uma matriz pode ser vista como um vetor bidimensional, como uma tabela com linhas e colunas, da mesma forma como na matemática, mas ela pode ter mais dimensões (multidimensional)
- Uma matriz bidimensional é criada estaticamente com uma declaração do seguinte tipo:

```
int Mat[3][4];
```

- A matriz denominada Mat funciona como uma tabela organizada em 3 linhas (enumeradas de 0 a 2) e 4 colunas (enumeradas de 0 a 3), totalizando 12 elementos
- Cada elemento pode ser acessado por um índice na linha e outro na coluna, por exemplo:

```
Mat[2][3] ou Mat[0][2]
```

- Assim como nos vetores tradicionais, na declaração anterior, apenas o espaço de memória foi alocado, mas nenhum dado foi colocado nas posições da matriz
- Mas podemos declarar matrizes já com elementos dentro, como a seguir:

```
int Mat[3][4] = { {1, 7, 2, -1}, {8, 3, -5, 4}, {-2, 9, 3, -6} };
```

- Ou podemos atribuir valores durante a execução, conforme segue:

```
Mat[0][0] = 1;  
Mat[0][1] = 7;  
:      :      :  
Mat[2][2] = 3;  
Mat[2][3] = -6;
```

- Podemos visitar a matriz em todas as suas posições/elementos usando 2 laços aninhados, um para cada dimensão, para ler, imprimir ou operar sobre eles, conforme o seguinte exemplo:

```
for (lin=0; lin < 3; lin++)  
    for (col=0; col < 3; col++)  
        { printf("\nDigite Mat[\",lin,\"\",col,\"] = ");  
          scanf("%d",&Mat[lin][col]);  
          Mat[lin][col]=2*Mat[lin][col];  
        }
```

- Uma matriz também pode ser alocada dinamicamente, com malloc, para para isso precisamos definir um tipo para que o compilador entenda como as dimensões estão estruturada, conforme o seguinte exemplo:

```
typedef int tipoMatriz[50][50];
```

```
tipoMatriz *Mx;
```

```
Mx=malloc(sizeof(tipoMatriz));
```

- Essa alocação permite tratar a matriz da mesma forma que a alocação estática, exceto no acesso, pois deve-se usar `(*Mx)[i][j]`

- O modelo de alocação anterior aloca dinamicamente uma matriz de um tamanho pré-definido, mas é possível definir um tipo que permita alocar matrizes de dimensões diferentes dinamicamente, conforme a seguir, para uma matriz 10x20:

```
typedef int **tipoMatriz;  
tipoMatriz M1;  
  
M1=malloc(10*sizeof(int*));  
For (i=0;i<10;i++)  
    M1[i]=malloc(20*sizeof(int));
```

- Essa alocação permite tratar a matriz da mesma forma que a alocação estática, inclusive no acesso, o qual pode ser feito da mesma forma: `M1[i][j]`

Procedimentos e Funções



Enviando e recebendo matrizes dinâmicas nos parâmetros

```
○ typedef double
  tipoMatriz[maxDim][maxDim];

○ tipoMatriz *alocaMatriz1(void){
○   tipoMatriz *M;

○   M=malloc(sizeof(tipoMatriz));
○   return M;
○ }

○ Void alocaMatriz2(tipoMatriz **M){
○   int i,j;
○
○   *M=malloc(sizeof(tipoMatriz));
○ }

○ void main(void) {
    tipoMatriz *A, *B;

○   A=alocaMatriz1();
○   alocaMatriz2(&B);
○   :      :      :
○ }
```

- 1) Faça um programa para multiplicar uma matriz $A_{m \times n}$ por uma matriz $B_{n \times p}$ gerando a matriz $C_{m \times p}$, para números double. O usuário deve informar m , n e p . As matrizes A e B devem ser geradas aleatoriamente;
- Use somente alocação dinâmica para as matrizes, como tamanho máximo pré-definido/declarado e todas as funções/procedimentos devem passar parâmetros (não use diretamente as variáveis globais).
- Use variáveis com nomes sugestivos; crie um tipo para as matrizes, como uma dimensão máxima pré-definida/declarada;
- Crie/use as seguintes funções/procedimentos:
 - leDimensao: lê as dimensões de uma matriz e retorna os valores nos próprios parâmetros;
 - geraMatriz: aloca a matriz e gera aleatoriamente os elementos, de acordo com os parâmetros informados; não lê e nem imprime nada internamente; retorna o endereço da matriz alocada no nome da função;
 - mostraMatriz: imprime a matriz tabulada, no formato de tabela, de acordo com os parâmetros; o nome da matriz também deve ser passado como parâmetro;
 - multiplicaLinhaColuna: multiplica uma linha por uma coluna e retorna o valor no nome da função, conforme parâmetros informados;
 - multiplicaMatriz: multiplica as matrizes A e B , usando a função `multiplicaLinhaColuna`, e retorna a matriz resultante no parâmetro
- 2) Faça um segundo programa, com as mesmas funções/procedimentos, mas de forma a permitir alocar as matrizes do tamanho exato definido pelo usuário, não por um tamanho máximo pré-definido.